DISEÑO WEB EN ENTORNO CLIENTE

Eventos y trabajo con formularios

UT_0.4.2

CONTENIDOS

Cinco bloques en esta unidad

1

Eventos

de formulario, foco e interfaz

2

Delegación

un escuchador para muchos elementos

3

Acceso

a formularios y a sus campos

4

Envío

submit, reset y FormData

5

Validación

HTML5 y API de restricciones

Todos los ejemplos usan JavaScript ES6 y se cargan como módulo: type="module"

1

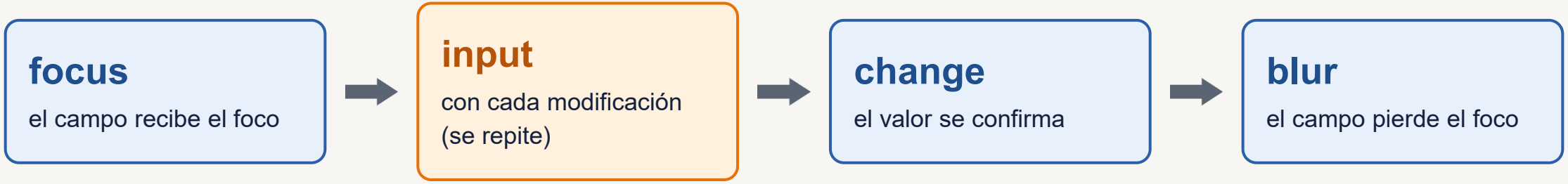
Eventos de formulario y de la interfaz

input, change, submit, foco, load y scroll

Eventos propios de los formularios

Evento	Cuándo se activa	Objeto
input	con cada modificación del valor	InputEvent
beforeinput	justo antes de modificarlo (cancelable)	InputEvent
change	el valor se confirma	Event
submit	se envía el formulario (se escucha en el <form>)	SubmitEvent
reset	se restablece el formulario	Event
invalid	un campo no supera la validación nativa	Event

Orden de los eventos de un campo



Al enviar el formulario (botón de envío o Intro) se activa **submit** en el <form>.

Ejemplo: al escribir «Ana» y salir del campo ocurre focus, input, input, input, change y blur.

input frente a change

```
const campo = document.getElementById('nombre');

// input: con cada modificación
campo.addEventListener('input', (evento) => {
  console.log(`Escribiendo: ${evento.target.value}`);
});

// change: cuando se confirma
campo.addEventListener('change', (evento) => {
  console.log(`Confirmado: ${evento.target.value}`);
});
```

input

cada tecla, al pegar o borrar

change

al confirmar: se pierde el foco o se pulsa Intro. En casillas y listas, al instante

submit: interceptar el envío

- Se escucha en el <form>, no en el botón
- preventDefault() evita el envío y la recarga
- evento.submitter es el botón que lo provocó
- FormData recoge los campos con atributo name

```
formulario.addEventListener('submit', (evento) => {
  evento.preventDefault();
  const datos = Object.fromEntries(
    new FormData(formulario)
  );
  console.log(datos);
  // { nombre: 'Ana', correo: '...' }
});
```

Eventos de foco

focus / blur

No burbujean

focusin / focusout

Sí burbujean: sirven para delegar

Si solo cambias estilos, CSS basta: **:focus** y **:focus-within**

```
formulario.addEventListener('focusin', (e) => {
  e.target.classList.add('activo');
});

formulario.addEventListener('focusout', (e) => {
  e.target.classList.remove('activo');
});
```

Eventos generales de la interfaz

Evento	Cuándo se activa
load	la página y todos sus recursos están cargados
DOMContentLoaded	el árbol DOM está completo
scroll	el usuario se desplaza
resize	cambia el tamaño de la ventana
beforeunload	justo antes de abandonar la página
visibilitychange	la pestaña se oculta o se muestra
cut · copy · paste	se corta, copia o pega en un campo
error	falla un recurso o se produce un error

unload está obsoleto: usa pagehide o visibilitychange.

load: la página ya está cargada

DOMContentLoaded

El HTML está listo. No espera a las imágenes.

load

La página y todos sus recursos están cargados.

```
window.addEventListener('load', () => {
  estado.textContent =
    `Cargada. Imágenes: ${document.images.length}`;
});
```

Con módulos ES6 el script ya se ejecuta con el DOM listo: no necesitas DOMContentLoaded.

scroll y su alternativa moderna

scroll

Se dispara muchísimas veces por segundo

```
window.addEventListener('scroll', () => {
  nombre.classList.toggle(
    'visible', window.scrollY > 10);
}, { passive: true });
```

IntersectionObserver

Avisa solo cuando cambia la visibilidad

```
const observador =
  new IntersectionObserver((entradas) => {
    entradas.forEach((entrada) => {
      if (entrada.isIntersecting) {
        entrada.target.classList.add('visible');
      }
    });
  }, { threshold: 0.2 });
observador.observe(elemento);
```

2

Delegación de eventos

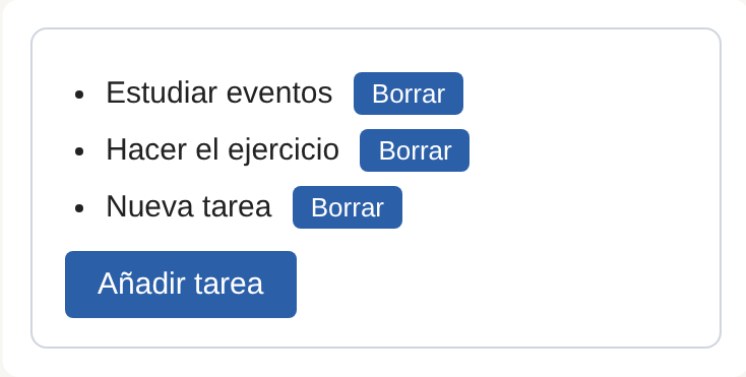
Un único escuchador para muchos elementos

2 · DELEGACIÓN

El problema: un escuchador por elemento

```
document.querySelectorAll('.borrar')
  .forEach((boton) => {
    boton.addEventListener('click', () => {
      boton.closest('li').remove();
    });
  });

agregar.addEventListener('click', () => {
  const tarea = document.createElement('li');
  // ... el botón nuevo NO tiene escuchador
  lista.append(tarea);
});
```



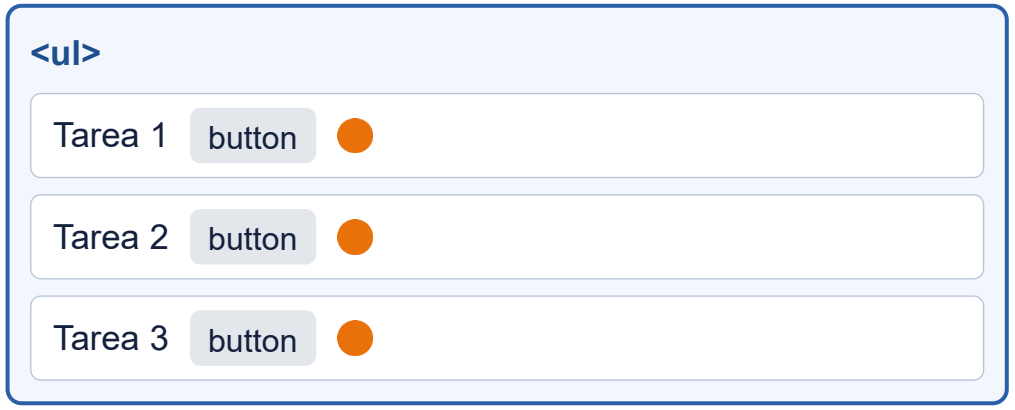
Las tareas añadidas después no responden al clic.

Y con mil elementos habría mil escuchadores.

2 · DELEGACIÓN

La idea: aprovechar el burbujeo

Sin delegación



Un escuchador por botón, y otro más por cada tarea nueva

Con delegación



El clic burbujea y lo atiende un solo escuchador, también para las tareas nuevas

2 · DELEGACIÓN

Seis pasos para delegar

1

Elegir el ancestro

Contiene a todos y existe desde el principio

2

Un escuchador

addEventListener en el ancestro

3

¿Quién fue?

evento.target es el elemento pulsado

4

Filtrar

target.closest('.borrar'); si es null, return

5

Comprobar

lista.contains(boton): ¿está dentro?

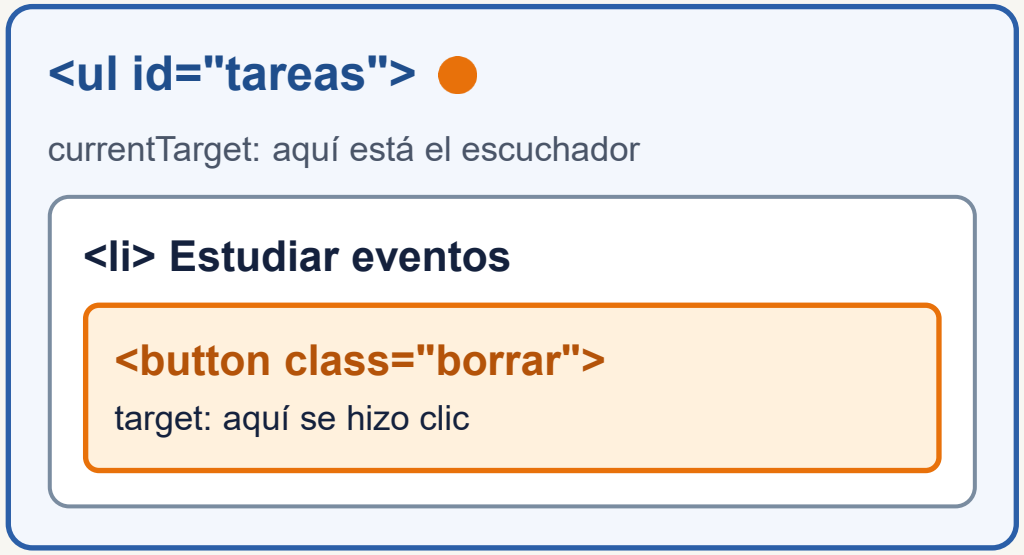
6

Actuar

boton.closest('li').remove()

2 · DELEGACIÓN

target y currentTarget



1. Clic en el botón: target es el botón
2. El evento burbujea por el li hasta el ul
3. Se ejecuta el escuchador del ul
4. closest('.borrar') localiza el botón
5. remove() elimina la tarea

2 · DELEGACIÓN

La solución con un único escuchador

```
const lista = document.getElementById('tareas');

lista.addEventListener('click', (evento) => {
  const boton = evento.target.closest('.borrar');
  if (!boton || !lista.contains(boton)) return;
  boton.closest('li').remove();
});
```

Pasos 4 y 5

filtrar y comprobar

Paso 6

actuar sobre la tarea

Las tareas nuevas funcionan sin registrar nada más.

¿Por qué closest() y no matches()?

```
<button class="borrar">
  <span>Borrar</span>
</button>
```

Si se pulsa el texto, evento.target es el , no el botón.

target.matches('.borrar')

false: el span no es el botón

target.closest('.borrar')

el <button>, se pulse donde se pulse dentro de él

Eventos que no burbujean

No burbujea	Alternativa
focus / blur	focusin / focusout
mouseenter / mouseleave	mouseover / mouseout
invalid	captura (true)
load / error	captura (true)

Escuchar en la fase de captura con el tercer argumento:

```
formulario.addEventListener('blur', (e) => {
  console.log(e.target.name);
}, true);
```

Errores frecuentes al delegar

Error	Solución
Usar solo evento.target	Usar closest()
No filtrar el elemento	if (!boton) return;
stopPropagation() en un hijo	El evento no llega al ancestro: evitarlo
Delegar todo en document	Elegir el ancestro más cercano
Delegar eventos sin burbujeo	focusin / focusout o captura
Delegar mousemove	Escuchador directo en el elemento

3

Acceso a formularios y campos

Seleccionar, leer valores y rellenar controles

3 · ACCESO

Tres formas de acceder a un formulario

getElementById() / querySelector()
Recomendado: no depende del orden

document.forms[0]
Depende de la posición del formulario

document.acceso
Su nombre puede chocar con propiedades de document



Los controles del formulario

```
const controles = formulario.elements;
controles[0].id;          // por posición
controles.usuario;      // por nombre o id (más legible)
controles.clave.value;  // su valor
```

Elemento	Tipo de objeto
input (text, password, radio, checkbox)	HTMLInputElement
textarea	HTMLTextAreaElement
select	HTMLSelectElement
option	HTMLOptionElement

3 · ACCESO

Leer el valor: value y this

```
campo.addEventListener('change', function () {
  console.log(this.value);
  // bien: this es el campo
});

campo.addEventListener('change', () => {
  console.log(campo.value);
  // bien: variable externa
  console.log(this.value);
  // error: en un módulo this es undefined
});
```

En una flecha, this no es el campo

Usa la variable o evento.target

value siempre es texto

Para números: valueAsNumber

Contraseña sin cifrar

value es texto plano

3 · ACCESO

Casillas y botones de opción

```
casilla.addEventListener('change', () => {
  console.log(casilla.checked);
  // true o false
});

// Botones de opción: un solo escuchador
formulario.addEventListener('change', () => {
  mensaje.textContent =
    formulario.elements.pizza.value;
});
```



El grupo tiene value: el valor del botón marcado.

3 · ACCESO

Listas de selección

```
lista.addEventListener('change', () => {
  const valor = lista.value;
  const indice = lista.selectedIndex;
  const opcion = lista.options[indice];
  mensaje.textContent =
    `${opcion.text} (${valor})`;
});
```



value no es el texto

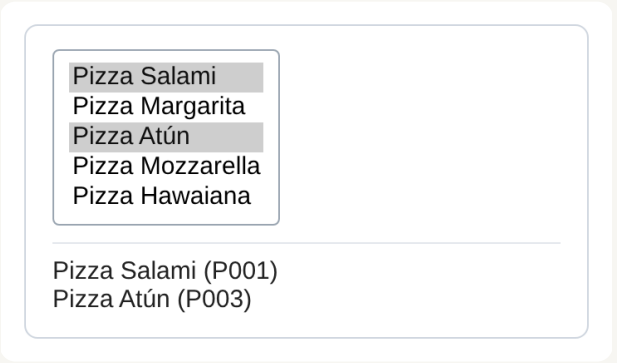
lista.value es P002; el texto está en opcion.text

3 · ACCESO

Selección múltiple y listas dinámicas

```
// select multiple: solo las opciones marcadas
for (const opcion of lista.selectedOptions) {
  console.log(opcion.text, opcion.value);
}

// Rellenar una lista con JavaScript
for (const pizza of pizzas) {
  lista.add(new Option(pizza.nombre, pizza.id));
}
```



selectedIndex solo devuelve la primera opción marcada: usa selectedOptions.

4 · ENVÍO

Enviar y restablecer con código

Forma de enviar	¿Valida?	¿Lanza submit?
Botón de envío o Intro	Sí	Sí
formulario.requestSubmit()	Sí	Sí
formulario.submit()	No	No

formulario.reset()

Vuelve a los valores iniciales del HTML, no necesariamente a vacío

Cuidado con name="submit"

Tapa el método: formulario.submit is not a function

4 · ENVÍO

FormData: leer el formulario de una vez

```
const datos = new FormData(formulario);

datos.get('nombre');       // 'Ana'
datos.getAll('extras');    // ['queso', 'aceitunas']
datos.append('origen', 'web');

Object.fromEntries(datos);
// solo el último de cada nombre

for (const [clave, valor] of datos) {
  console.log(`${clave} = ${valor}`);
}
```

get() y getAll()
un valor, o todos los de un mismo nombre

append, set, delete
modifican los datos antes de usarlos

Es iterable
for...of recorre los pares [clave, valor]

No recoge: campos sin name, deshabilitados, casillas o radios sin marcar y botones.

5

Validación de formularios

Validación nativa de HTML5 y API de restricciones

Validación nativa de HTML5

Atributo	Qué exige
required	campo obligatorio
type="email" / "url"	formato de correo o URL
min / max	rango numérico
minlength / maxlength	longitud del texto
pattern	expresión regular

Sin una línea de JavaScript el navegador bloquea el envío y muestra un aviso.



Aviso nativo (representación)

5 · VALIDACIÓN

Pseudoclases CSS de validación

:invalid / :valid

Se aplican desde que carga la página: el formulario aparece lleno de campos en rojo antes de escribir.

:user-invalid / :user-valid

Solo después de que el usuario interactúe con el campo. Recomendado.

```
input:user-invalid { border: 2px solid red; }
input:user-valid   { border: 2px solid lightgreen; }
```

5 · VALIDACIÓN

API de validación de restricciones

Propiedades

willValidate

¿se valida el campo?

validity

objeto con la causa del error

validationMessage

mensaje del navegador (" si es válido)

Métodos

checkValidity()

¿es válido? (un campo o todo el formulario)

reportValidity()

igual, y además muestra el aviso

setCustomValidity(texto)

mensaje propio (" = válido)

validity: la causa del error

Propiedad	Es true cuando...
valueMissing	falta un valor obligatorio
typeMismatch	el formato no es el exigido (correo, URL)
patternMismatch	no cumple el pattern
tooShort / tooLong	es demasiado corto o demasiado largo
rangeUnderflow / rangeOverflow	está por debajo del mínimo o por encima del máximo
stepMismatch	no cumple step
badInput	el navegador no puede interpretar lo escrito
customError	hay un error de setCustomValidity()
valid	no hay ningún error

Mostrar el mensaje bajo el campo

```
<p id="error-correo" class="error"></p>

.error:empty { display: none; }

campoCorreo.addEventListener('change', () => {
  zonaError.textContent =
    campoCorreo.validationMessage;
});
```

validationMessage es '' si el campo es válido: no hace falta un if.



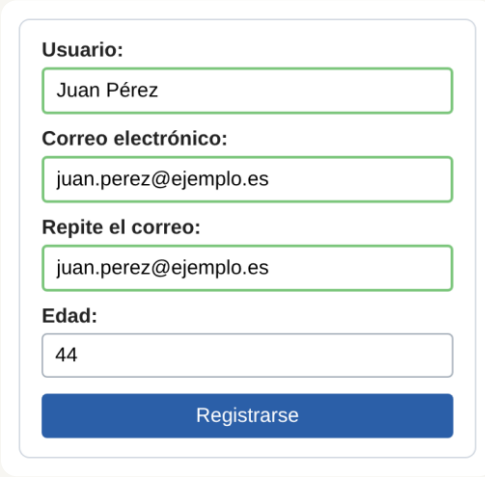
Validación propia: setCustomValidity

```
function comprobarCorreos() {
  const mensaje = correo.value === correo2.value
    ? '' : 'Los correos deben coincidir.';
  correo2.setCustomValidity(mensaje);
  zonaError2.textContent =
    correo2.validationMessage;
}
correo.addEventListener('input', comprobarCorreos);
correo2.addEventListener('input', comprobarCorreos);
```

No olvides limpiar: sin setCustomValidity('') el campo sigue siendo inválido.



No coinciden



Coinciden

Controlar cuándo y cómo se muestran los errores

```
formulario.noValidate = true;

formulario.addEventListener('submit', (evento) => {
  if (!formulario.checkValidity()) {
    evento.preventDefault();
  }
});

// invalid no burbujea: se captura
formulario.addEventListener('invalid', (evento) => {
  evento.preventDefault();
  mostrarError(evento.target);
}, true);
```

noValidate

el navegador no muestra sus avisos

checkValidity()

comprueba todo el formulario

invalid

no burbujea: captura (true)

Ejemplo completo con módulos ES6

validaciones.js

```
export function mostrarError(campo) {
  const zona = document
    .getElementById(`error-${campo.id}`);
  if (zona) {
    zona.textContent =
      campo.validationMessage;
  }
}
```

registro.js

```
import { mostrarError } from './validaciones.js';

// focusout burbuja: un escuchador para todos
formulario.addEventListener('focusout', (e) => {
  if (e.target.willValidate) mostrarError(e.target);
});

formulario.addEventListener('submit', (e) => {
  if (!formulario.checkValidity()) {
    e.preventDefault();
    for (const campo of formulario.elements) {
      if (campo.willValidate) mostrarError(campo);
    }
  }
});
```

RESUMEN

Lo que debes llevarte

Eventos

input, change, submit y focusin / focusout

Delegación

Un escuchador en el ancestro, target y closest()

Acceso

elements, value, checked y selectedOptions

Envío

requestSubmit(), preventDefault() y FormData

Validación HTML

required, type, min, max, pattern y :user-invalid

Validación JS

validity, validationMessage y setCustomValidity()